목진희

AI·자동화 엔지니어

경력기술서

jinheemok815@gmail.com
010.9030.2888
jin-tonix.github.io
서울시 강남구 역삼동

1. 스타일셀러(StyleSeller) | AI·자동화 PL

2025.11 ~ 재직 중

비즈니스팀에서 직원들이 반복적으로 처리하던 거래처 응대, 기록 정리, 가격 계산, 상품 등록 등의 업무를 자동화하고 있습니다. 업무를 관찰해 자동화 대상을 정하고, 요구사항 정리부터 시스템 설계, 구현, 배포 및 운영까지 담당합니다.

프로젝트 1. 에이전트 플랫폼 및 스킬 라우터(Spark)

2026.05 ~ 현재

Situation | 배경

에이전트를 여러 개로 나누어 운영하면서 업무를 전달할 때마다 맥락이 끊기고, 각각의 에이전트를 추적하고 관리하는 데 시간이 들었습니다. 또한 공용 스킬이 115개까지 늘어나면서 LLM이 전체 목록을 확인해 적절한 스킬을 선택하는 데 시간과 비용이 커지는 문제가 있었습니다.

Action | 액션

- Paperclip을 포크한 사내 에이전트 플랫폼의 작업지시서와 에이전트 위임 구조 설계 및 구현
- 위임 깊이 2단계, 작업지시서당 최대 5회 위임 제한 적용
- 도구 사용 승인을 프롬프트가 아닌 서버와 DB에서 강제하도록 구현
- BM25 검색으로 전체 스킬 중 관련 후보를 먼저 추출하고, 판정 모델 Jev가 게이트와 적합도를 판단한 뒤 Gemini가 최종 선택하는 구조 설계
- 평가 데이터 132건을 구성하고 정답표 오류와 가짜 스킬 1개를 찾아 수정한 뒤 재측정

Result | 결과

- 평가 132건 중 119건의 정답을 찾아 정답률 90.15% 달성(평가셋은 도입 당시 신규였고, 이후 튜닝에 재사용)
- Gemini가 확인하는 스킬 수를 115개에서 5개로 축소
- Gemini 호출 비용은 기존 대비 약 1/10 수준으로 감소할 것으로 추정(단가와 줄어든 입력량 기준 계산, 실측 아님)
- 개선한 라우팅 구조를 하위 에이전트에 전면 적용
- 동시 요청 10건에서도 도구 승인이 중복 실행되지 않도록 구현
- 하루 유료 API 호출 비용 3만 원 상한 적용

기술: Paperclip, Express, Drizzle, PostgreSQL, MCP SDK, Hermes, Python, FastAPI, LiteLLM, Redis, Langfuse, Gemini Flash

프로젝트 2. 벤더 CS 에이전트

2026.06 ~ 현재

Situation | 배경

거래처 문의가 직원들의 카카오톡 단체 채팅방으로 들어오고 있었지만, 공식 API가 없어 대화 수집과 자동 응대가 어려웠습니다. 답변에는 금액, 수수료, 브랜드명 등 잘못 전달하면 안 되는 정보가 많아 단순히 LLM이 생성한 답변을 그대로 발송하기도 어려웠습니다.

Action | 액션

- 직원 PC에서 카카오톡 대화를 수집해 서버로 전달하는 Windows 수집기 구현
- kakaocli를 포크해 직원 Mac에서 답변을 발송하고 도착 여부를 확인하는 구조 구현
- 채팅방 이름과 응대 대상 거래처 명단을 대조해 거래처가 확인되지 않으면 답변을 생성하거나 발송하지 않도록 설계
- 거래처 기록과 사내 지식 허브를 검색해 답변 초안 생성
- 초안에 포함된 금액, 비율, 브랜드명을 근거 자료 및 코드와 대조하고, 불일치가 있으면 재검색 후 수정하도록 검증 단계 구현
- 중복 메시지 수집과 macOS 발송 오류를 추적해 운영 안정성 개선

Result | 결과

- 메시지 1건이 51번 중복 수집되던 문제를 15초 중복 제거 방식으로 해결
- 기존 발송 147건 중 31건(21%)이 실패하던 macOS 접근성 멈춤 문제를 3초 제한 시간과 카카오톡 DB 도착 확인 방식으로 수정
- 실제 거래처 응대에 활용하는 에이전트 운영 중
- 프로젝트 관련 커밋 600개, 테스트 파일 107개(2026.09.23 기준)

기술: Swift, kakaocli, Python, LightRAG, Gemini Flash, Google Sheets, Google Drive

프로젝트 3. 거래처 현황판 및 운영 원장

2026.10.19 ~ 2026.10.23 구현, 운영 중

Situation | 배경

거래처와 주고받은 기록이 카카오톡, 주문 메일함, 슬랙, 메일 첨부 엑셀 주문서 등에 흩어져 있었습니다. 담당 직원이 거래처별 진행 상황을 확인하려면 여러 채널을 직접 확인해야 했고, 동일한 거래 기록이 여러 곳에 남는 문제도 있었습니다.

Action | 액션

- 거래처와 기록을 연결하기 위한 원장 데이터 구조 및 매칭 규칙 설계
- 카카오톡, 메일, 슬랙, 엑셀 주문서의 데이터를 수집하는 인입 API와 메일 수집기 구현
- 거래처별 진행 상황을 확인할 수 있는 화면 탭 6개 구현
- 수집한 기록을 하루 단위로 요약해 거래처별 달력에 표시
- LLM을 사용하지 않고 스레드, 사업자번호, 카카오톡 방, 이메일 순서로 거래처를 매칭하도록 구현
- 매칭 키가 불일치하거나 확실하지 않은 기록은 자동 연결하지 않고 미분류 상태로 두어 직원이 직접 확인하도록 설계

Result | 결과

- 수백 곳 거래처의 거래 단계를 한 화면에서 확인할 수 있도록 구현
- 평일 하루 약 100~200건의 거래 기록 자동 수집
- 5일 만에 구현하고 실제 운영에 적용
- 매일 18시 5분에 최대 500여 개 거래처의 요약 정보 갱신
- 하루 AI 비용이 2,000원을 초과하면 자동으로 배치를 중단하도록 비용 제한 적용

기술: React, Tailwind CSS, Vitest, Python, Pydantic, PostgreSQL, Gemini Flash

프로젝트 4. 사내 업무툴 및 가격·상품·거래처 매칭 자동화

2026.03 ~ 현재

Situation | 배경

가격 계산이 구글시트 수식에 의존하고 있었으며, 공급사 카탈로그에서 상품을 확인해 등록하거나 셀러에게 맞춤형 카카오톡을 발송하는 작업도 직원이 직접 처리하고 있었습니다.

Action | 액션

- 기존 구글시트의 가격 계산 수식을 코드로 옮기고 올림 단위까지 동일하게 적용
- LLM은 수정이 필요한 입력값을 JSON으로 반환하고, 실제 금액 계산은 코드에서 다시 수행하도록 분리
- PDF, 이미지, 엑셀, URL 형태의 상품 카탈로그에서 LLM이 상품 후보 정보를 추출하도록 구현
- 추출된 상품을 직원이 확인한 경우에만 실제 카탈로그에 등록하도록 설계
- CRM에서 전달받은 대상자에게 맞춤형 카카오톡을 발송하고, 500명 이상 발송 시 승인 대기 상태로 전환
- 직원 PC에서 셀러 찾기 작업을 수행하는 워커 구현

Result | 결과

- 가격 계산 테스트 43개 케이스 통과
- 일부 실제 상품의 계산 결과를 구글시트와 항목별로 대조해 검증
- 맞춤형 카카오톡 하루 261명 발송(2026.09.22 기준)
- 셀러 찾기 워커를 직원 PC 9대에서 운영(2026.09.18 기준, 등록된 직원 PC는 11대·2026.09.16 기준)
- 여러 PC에서 공유하던 인증 키를 PC별 개별 키로 변경

기술: React, Vitest, PostgreSQL, PostgREST, Gemini Flash, RAG, kakaocli

프로젝트 5. LightRAG 지식 허브 및 메일 에이전트

2026.03.31 ~ 현재

Situation | 배경

여러 업무 에이전트가 동일한 회사 내부 정보를 참고해야 했습니다. 공용 메일함으로 들어오는 거래처 문의에도 단순한 답변 생성이 아니라 실제 업무 자료를 근거로 한 답변 초안이 필요했습니다.

Action | 액션

- 여러 에이전트가 공통으로 사용하는 워크스페이스 등록부와 지식 검색 구조 설계
- 검색 계획, 검색 결과의 근거 판정, 답변 초안 검증 순서 구현
- 메일 문의에 대해 사내 지식과 거래처 자료를 검색하고 담당자 Gmail 초안함에 답변 초안을 생성하는 메일 에이전트 구현

- 생성된 답변의 주장 최대 6개를 근거 자료와 대조하도록 검증 로직 구현
- 근거가 없는 주장이 발견되면 다시 작성하고, 재작성 후에도 근거가 없는 내용이 남으면 초안 자체를 폐기하도록 설계
- 발송은 자동으로 진행하지 않고 담당 직원이 최종 확인하도록 구성

Result | 결과

- 총 4개 에이전트가 13개 워크스페이스의 지식을 공유
- 워크스페이스 이름이 잘못 연결되면 서버가 실행되지 않도록 검증 구조 적용
- 보낸 메일 1,943건 중 48건이 학습 과정에서 누락되던 문제를 커서 구간을 겹치고 늦춰 읽는 방식으로 바꿔 해결
- 담당 직원이 초안을 검토한 뒤 발송하는 메일 응대 흐름 운영

기술: Python, FastAPI, Pydantic, LightRAG, PostgreSQL, pgvector, Neo4j, FlashRank, Gemini Flash, Gmail, pytest

프로젝트 6. 콘텐츠 오토파일럿

2026.10.03 ~ 현재

Situation | 배경

회사는 약 1년 전 콘텐츠 제작 자동화를 시도했으나, 당시 영상 생성 모델의 품질 문제로 완전 자동화까지 구현하지 못하고 반자동화로 마무리했습니다. 이후 모델의 생성 품질이 개선되면서 제가 다시 완전 자동화를 맡게 됐습니다.

Action | 액션

- 실무자가 사내 업무툴에서 릴스 제작을 요청하면 대본 생성, 영상 렌더링, 미리보기 승인으로 이어지는 프로세스 구현
- 9:16 비율의 MP4 영상 생성 및 Google Drive 저장 구조 구현
- LLM이 아닌 상태기계가 각 제작 단계의 진행을 관리하도록 설계
- 게시 기본값은 수동 승인, 반복 자동화는 dry-run, 유료 생성 기능은 비활성화하도록 구성
- 생성 콘텐츠에 AI 생성 콘텐츠 표시가 자동으로 붙도록 구현
- 게시 전 검수와 승인 단계를 분리

Result | 결과

- 1080×1920 해상도의 MP4 영상을 생성해 Drive에 저장하는 운영 canary 검증 완료
- 신규 영상 제작 경로는 프롬프트 품질을 개선 중이며 SNS 실게시 전 단계
- 기존 영상에 계정별 캡션과 해시태그를 적용해 여러 계정에 동시 게시하는 별도 경로는 실제 운영 중
- 계정별 하루 게시 1건 제한을 DB 인덱스로 강제
- 관련 서비스 3개의 테스트 파일 총 617개

기술: Python, Gemini Flash, Hermes, HyperFrames, ffprobe, Google Drive, Supabase, PostgreSQL

프로젝트 7. 직원 PC 배포 및 운영 환경 구축

2026.02 ~ 현재

Situation | 배경

카카오톡 수집, 인스타그램 DM 작업, 셀러 찾기 등 여러 자동화 작업을 직원들의 Mac과 Windows PC에서 실행해야 했습니다. PC마다 직접 설치하고 관리하는 방식으로는 업데이트와 인증 관리가 어려웠고, 직원이 늘어날수록 운영 부담도 커지는 상황이었습니다.

Action | 액션

- 한 줄 명령으로 설치할 수 있는 Mac·Windows 설치기 구현
- 1회용 토큰을 이용한 기기별 인증 및 개별 키 발급 구조 설계
- Mac 도구와 Windows CS 도구는 PC마다 갱신 시점을 다르게 설정하는 자동 업데이트 구조 구현(별도 Windows 수집기는 재설치로 갱신)
- 서버는 커밋된 이미지가 확인된 경우에만 배포하고, 이미지가 없으면 배포를 중단하도록 안전장치 적용
- Slack 감시 크론을 통해 서비스 이상 여부 확인
- 클라우드 DB 사용 한도 초과로 업무툴 API가 중단됐을 때 대체 서버 구축 및 복구

Result | 결과

- 직원 Mac·Windows PC 총 11대에 한 줄 명령 방식으로 설치 및 배포(2026.09.16 등록 기준)
- 2026.08.20 클라우드 DB 사용 한도 초과로 업무툴 API가 중단됐을 때 VM에 동일한 PostgREST API를 구축해 당일 복구
- 앱 내부의 DB 호출 코드 100여 곳을 수정하지 않고 서비스 복구
- 2026.08.06 ~ 08.25 기간에 DB 4개를 자체 호스팅 환경으로 이전

기술: macOS Bash, Windows PowerShell, launchd, Windows 작업 스케줄러, Node.js, Python, PostgreSQL, PostgREST, NGINX, Docker, GCP, Slack

2. 위리드(We:Review) | AI 개발 인턴

2025.08 ~ 2025.10

프로젝트 8. 영수증 검수 자동화 파이프라인

Situation | 배경

영수증 이미지에서 상품명과 가격을 추출하고 확인하는 작업을 사람이 직접 수행하고 있었습니다. 영수증이 길거나 글자가 흩어져 있는 경우 OCR 결과를 그대로 사용하기 어려웠고, 상품과 가격을 다시 확인하는 데 많은 시간이 소요됐습니다.

Action | 액션

- PaddleOCR를 이용해 영수증 이미지를 타일 단위로 분할하고 텍스트 추출
- DBSCAN으로 흩어진 OCR 결과를 분석해 줄 단위로 복원
- Qwen 계열 오픈 LLM에 few-shot 방식을 적용해 상품명과 가격을 JSON 구조로 추출
- Python과 FastAPI를 활용해 검수 파이프라인 서비스화

Result | 결과

- 검수 정확도 70%에서 95%로 향상
- 검수 시간 85% 단축(직접 측정)
- OCR부터 LLM 구조화 데이터 추출까지 연결한 검수 자동화 파이프라인 구현 및 서비스화 참여

기술: Python, PaddleOCR, DBSCAN, Qwen 계열 LLM, FastAPI

3. 이전 경력

전원어린이집·방과후학교 | 운영실장 2015.03 ~ 2018.02
교사 채용, 연간 커리큘럼 및 행사 기획, 학부모·운영위원회 소통, 원장 대행 운영 관리.

정상어학원·파츠앤팬스·몬테키즈어학원 | 영어 강사 2018.08 ~ 2024.02
수업 설계, 교안 제작, 학생별 학습 계획 수립, 학부모 상담 및 학습 관리.